

学与成设计网

Next.js 全栈重构 · 架构设计方案

技术栈: Next.js 16 + TypeScript + Prisma + PostgreSQL + Tailwind CSS 4

文档版本: v1.0 · 2026-05-18

一、项目核心目标

一句话定义：把旧站 (PHP LzCMS) 的 SKP 模型 / 教程 / 材质 / 软件下载四种核心资产，重构为可规模化运营的 Next.js 全栈内容平台。

业务目标

- 日均 5000+ 访客的内容消费体验
- 后台单人每天录入 20+ 条资源
- 支持会员 / VIP 分层下载权限
- SEO 可索引所有资源页

技术目标

- SQL 注入 / XSS / CSRF 全量防御
- 图片 OSS 化，服务器零存储压力
- API 响应 < 200ms (SSG + ISR)
- 批量上传 1 分钟完成 1 条资源

二、信息架构

2.1 前台地图

```
首页 (/)
├─ 搜索 (/search?q=xxx)
├─ 模型 (/models)
│   ├─ 建筑SU模型 (/models/architecture)
│   ├─ 工装SU模型 (/models/commercial)
│   ├─ 家装SU模型 (/models/residential)
│   ├─ 园林SU模型 (/models/landscape)
│   └─ D5模型 (/models/d5)
├─ 教程 (/courses)
│   ├─ D5专题 (/courses/d5)
│   ├─ SketchUp (/courses/sketchup)
│   ├─ Layout (/courses/layout)
│   └─ Enscape (/courses/enscape)
├─ 材质 (/materials)
│   ├─ PBR贴图 (/materials/pbr)
│   └─ HDR贴图 (/materials/hdr)
├─ 软件 (/software)
│   ├─ SketchUp (/software/sketchup)
│   └─ Enscape (/software/enscape)
├─ 资源详情 (/resource/[slug])
├─ 会员中心 (/member)
│   ├─ 我的收藏 (/member/favorites)
│   ├─ 下载记录 (/member/downloads)
│   └─ 浏览历史 (/member/history)
└─ 登录 (/login)
```

2.2 后台地图

```
管理后台 (/admin)
├─ 工作台 (/admin)          - 概览统计 + 快捷入口
├─ 内容管理
│   ├─ 资源列表 (/admin/resources)    - 全部 ContentItem 列表
│   ├─ 新增资源 (/admin/resources/new) - 单个 / 批量录入
│   └─ 编辑资源 (/admin/resources/[id])
├─ 分类管理 (/admin/categories)      - 树形分类 CRUD
├─ 标签管理 (/admin/tags)
├─ 文件管理 (/admin/files)           - ResourceFile 维护
├─ 用户管理 (/admin/users)
├─ 会员管理 (/admin/members)        - 会员权益配置
└─ 系统设置 (/admin/settings)
```

三、数据库设计 (扩展现有 Prisma Schema)

3.1 现有模型评价

当前 Prisma schema 已有 **User**、**Category**、**Tag**、**ContentItem**、**ResourceFile** 五个核心模型，结构合理，覆盖了 80% 需求。以下仅标记需新增/调整的字段和表。

3.2 需新增的模型

```
model User {
  // 新增字段
  phone      String?          // 手机号（可选绑定）
  membership MembershipLevel @default(FREE)
  points      Int              @default(0)    // 积分
  signInAt    DateTime?        // 最后签到时间
  bannedAt    DateTime?        // 封禁时间
  favorites    Favorite[]      // 收藏
  downloads   DownloadLog[]    // 下载记录
  history      BrowseHistory[] // 浏览历史
}

model Favorite {
  id          String          @id @default(cuid())
  userId      String
  contentItemId String
  createdAt   DateTime        @default(now())
  user         User            @relation(fields: [userId], references: [id])
  contentItem  ContentItem     @relation(fields: [contentItemId], references: [id])
  @@unique([userId, contentItemId])
}

model DownloadLog {
  id          String          @id @default(cuid())
  userId      String
  contentItemId String
  fileId       String?
  downloadedAt DateTime        @default(now())
  user         User            @relation(fields: [userId], references: [id])
  contentItem  ContentItem     @relation(fields: [contentItemId], references: [id])
}

model BrowseHistory {
  id          String          @id @default(cuid())
  userId      String
  contentItemId String
  viewedAt    DateTime        @default(now())
  user         User            @relation(fields: [userId], references: [id])
  contentItem  ContentItem     @relation(fields: [contentItemId], references: [id])
}

model AuditLog {
  id          String          @id @default(cuid())
  userId      String
```

```

    action      String          // CREATE / UPDATE / DELETE / LOGIN
    targetType   String?         // ContentItem / User / Category
    targetId     String?
    detail       String?
    createdAt    DateTime @default(now())
}

enum MembershipLevel {
    FREE
    MEMBER
    VIP
}

```

3.3 ContentItem 需补充字段

```

model ContentItem {
    // 新增
    requiredPoints Int          @default(0)    // 所需积分
    sortOrder      Int          @default(0)    // 排序
    isSticky       Boolean      @default(false) // 置顶
    isRecommended  Boolean      @default(false) // 推荐
    coverImage     String?      // 封面图 → 拆到 Media 模型更佳
    images         Media[]      // 内容图片 (1:N)
}

model Media {
    id             String       @id @default(cuid())
    contentItemId  String
    url            String       // OSS / 本地路径
    type          MediaType     // COVER / CONTENT / THUMBNAIL
    sortOrder      Int          @default(0)
    contentItem    ContentItem @relation(fields: [contentItemId], references: [id])
}

enum MediaType {
    COVER
    CONTENT
    THUMBNAIL
}

```

四、路由与页面设计

4.1 页面分三类策略

策略	适用页面	原因
SSG	首页、分类列表、资源详情	内容变化频率低（一天几次），静态生成速度最快
ISR	搜索结果、热门排行	每小时 / 每天自动更新一次即可
SSR	会员中心、收藏/下载记录	依赖登录态，必须服务端实时渲染

4.2 文件目录结构

```
src/app/
├─ layout.tsx           # 根布局 (Geist 字体、元数据)
├─ page.tsx             # 首页 (SSG)
├─ loading.tsx / error.tsx # 全局加载/错误
├─ not-found.tsx        # 404
├─ login/
│   └─ page.tsx         # 登录页 (SSR)
├─ search/
│   └─ page.tsx         # 搜索结果 (ISR: 1h)
├─ models/
│   └─ page.tsx         # 模型总览
│       └─ [category]/
│           └─ page.tsx # 分类列表 (SSG + generateStaticParams)
├─ courses/
│   └─ [category]/page.tsx
├─ materials/
│   └─ [category]/page.tsx
├─ software/
│   └─ [category]/page.tsx
├─ resource/
│   └─ [slug]/
│       └─ page.tsx     # 资源详情 (SSG + 增量再验证)
├─ member/
│   └─ page.tsx         # 会员中心 (SSR)
│   └─ favorites/page.tsx
│   └─ downloads/page.tsx
│   └─ history/page.tsx
├─ admin/
│   └─ layout.tsx       # 后台布局 (侧栏 + 鉴权)
│   └─ page.tsx         # 工作台
│   └─ resources/
│       └─ page.tsx     # 列表
│       └─ new/
│           └─ page.tsx # 新增 (单条 / 批量)
│           └─ [id]/
│               └─ page.tsx # 编辑
│   └─ categories/page.tsx
│   └─ tags/page.tsx
│   └─ users/page.tsx
│   └─ settings/page.tsx
└─ api/
    └─ auth/
        └─ login/route.ts
        └─ register/route.ts
```



```
|   └─ logout/route.ts
└─ resources/
|   └─ route.ts           # GET 列表 / POST 新增
|   └─ [id]/route.ts      # PATCH 编辑 / DELETE 删除
└─ media/
|   └─ upload/route.ts    # 封面/内容图上传
└─ favorites/route.ts    # 收藏 toggle
└─ search/route.ts       # 全文搜索
└─ batch/
    └─ route.ts           # 批量导入接口
```

五、组件架构

5.1 层级划分

```

src/components/
├─ ui/                                # 原子 UI 组件（设计系统）
│   ├─ button.tsx
│   ├─ input.tsx
│   ├─ select.tsx
│   ├─ badge.tsx
│   ├─ card.tsx
│   ├─ dialog.tsx                    # 弹窗
│   ├─ dropdown.tsx
│   ├─ table.tsx
│   └─ toast.tsx
├─ shared/                            # 业务通用组件
│   ├─ resource-card.tsx             # 资源卡片（已有，需改造）
│   ├─ category-tree.tsx            # 分类树
│   ├─ media-uploader.tsx           # 图片上传
│   ├─ search-bar.tsx               # 搜索框
│   ├─ pagination.tsx
│   └─ empty-state.tsx
├─ layout/                            # 布局组件
│   ├─ site-header.tsx              # 前台头部（已有）
│   ├─ site-footer.tsx              # 前台底部（已有）
│   ├─ admin-sidebar.tsx            # 后台侧栏
│   └─ admin-header.tsx             # 后台顶栏
├─ home/                              # 首页专用
│   ├─ hero-section.tsx             # 已有
│   ├─ quick-stats.tsx              # 已有
│   ├─ topic-strip.tsx              # 已有
│   ├─ resource-section.tsx         # 已有
│   └─ member-section.tsx           # 已有
├─ resource/                          # 资源详情页专用
│   ├─ resource-detail.tsx          # 详情主体
│   ├─ resource-sidebar.tsx         # 相关推荐
│   └─ download-panel.tsx           # 下载面板
└─ admin/                             # 后台专用
    ├─ resource-form.tsx            # 资源录入表单（核心）
    ├─ resource-table.tsx           # 资源列表表格
    ├─ batch-upload.tsx             # 批量上传面板
    ├─ category-manager.tsx         # 分类管理
    └─ stats-dashboard.tsx          # 统计仪表盘

```

5.2 批量上传表单设计（核心组件）

resource-form.tsx 设计思路

根据参考文档，后台录入一条资源的字段固定。表单组件应支持两种模式：

- 1. 单条模式：常规表单，逐个字段填写
- 2. 批量模式：上传一个 TXT（含标题+链接+密码）+ 选择图片文件夹 → 自动解析并逐条提交

字段对照：

文档字段	Prisma 字段	组件
标题	ContentItem.title	<Input>
封面图	Media(type=COVER)	<MediaUploader>
下载地址	ResourceFile.fileUrl	<Input>
下载密码	ResourceFile.password	<Input>
所需积分	ContentItem.requiredPoints	<Input type="number">
首页分类	Category.id	<Select>
内容图片	Media(type=CONTENT)	<MediaUploader multiple>
排序	ContentItem.sortOrder	<Input type="number">

六、认证与权限模型

6.1 三端用户体系

角色	权限	登录方式
访客（未登录）	浏览公开资源、搜索	无需登录
会员（MEMBER）	下载公开+会员资源、收藏、查看历史	邮箱+密码 / 微信扫码
VIP	全部资源可下载、无需积分	继承会员登录
编辑（EDITOR）	新增/编辑内容、上传文件	邮箱+密码+OTP
管理员（ADMIN）	全权限：用户管理、系统设置、删除	邮箱+密码+OTP

6.2 鉴权中间件

```
// src/middleware.ts
import { NextRequest, NextResponse } from 'next/server';
import { getToken } from 'next-auth/jwt';

export async function middleware(req: NextRequest) {
  const { pathname } = req.nextUrl;

  // 后台路由保护
  if (pathname.startsWith('/admin')) {
    const token = await getToken({ req, secret: process.env.AUTH_SECRET });
    if (!token || (token.role !== 'ADMIN' && token.role !== 'EDITOR')) {
      // 不暴露后台是否存在 → 统一 404 而非 302
      return NextResponse.rewrite(new URL('/404', req.url));
    }
  }

  // 会员路由保护
  if (pathname.startsWith('/member')) {
    const token = await getToken({ req, secret: process.env.AUTH_SECRET });
    if (!token) {
      return NextResponse.redirect(new URL('/login?redirect=' + pathname, req.url));
    }
  }

  // 安全头（全站）
  const response = NextResponse.next();
  response.headers.set('X-Frame-Options', 'SAMEORIGIN');
  response.headers.set('X-Content-Type-Options', 'nosniff');
  response.headers.set('Referrer-Policy', 'strict-origin-when-cross-origin');
  response.headers.set(
    'Content-Security-Policy',
    "default-src 'self'; img-src 'self' data: https;; style-src 'self' 'unsafe-inline'; script
  );
  return response;
}

export const config = {
  matcher: ['/admin/:path*', '/member/:path*', '/((?!api|_next/static|_next/image|favicon.ico)
};
```

七、数据流设计

7.1 前台浏览流程



7.2 下载鉴权流程

```
用户点击「下载」
↓
GET /api/resources/[id]/download
↓ Session 校验
├─ 未登录 → 返回 401, 前端弹登录框
├─ role=FREE, visibility=MEMBER/VIP → 返回 403, "会员专享"
├─ role=MEMBER, visibility=VIP → 返回 403, "VIP专享"
└─ 权限通过 → 记录 DownloadLog (去重) → 返回 ResourceFile.fileUrl + password
    → 前端展示网盘链接和提取码 (复制按钮)
```

7.3 后台批量上传流程 (替换手动截图模式)

旧流程 vs 新流程

步骤	旧方式	新方式
1. 进入后台	打开 /admin/index/index.html	打开 /admin/resources/new → 选批量模式
2. 上传链接文件	手动解析 txt	拖拽或选择 baidu.txt → 自动解析标题+链接+密码
3. 上传封面	Windows 文件选择框	拖拽或选择 封面-N.jpg → 组件内预览 + 进度条
4. 上传内容图	富文本编辑器图片按钮	拖拽或选择 N.jpg → 自动关联
5. 填积分/分类/排序	手动输入	批量模式：默认 0 / SU模型 / 自动递增
6. 提交	点击"立即提交"	点击"提交" → API → 成功 toast → 自动载入下一条

```
// API: POST /api/batch 批量导入
// 请求体:
{
  "items": [{
    "title": "SU-21 现代图书馆室内设计",
    "coverFile": "封面-21.jpg",          // 前端已上传后返回的 key
    "contentFile": "21.jpg",
    "downloadUrl": "https://pan.baidu.com/s/xxxx",
    "downloadPassword": "abcd",
    "requiredPoints": 0,
    "categorySlug": "su-models",
    "sortOrder": 78
  }]
}

// 响应:
{
  "success": true,
  "created": 1,
  "failed": 0,
  "nextSortOrder": 79
}
```

八、图片 / 文件存储方案

推荐：阿里云 OSS / Cloudflare R2

封面图和内容图不再存服务器本地 /uploads/，而是：

1. 前端直传 OSS：用 STS 临时凭证，图片从浏览器直接上传到 OSS，不经过 Next.js 服务器

2. 回调写入数据库：OSS 上传成功回调 → API 写入 Media 表

3. CDN 加速：图片 URL 走 CDN，支持缩略图参数 (?x-oss-process=image/resize,w_320)

4. 私有资源保护：会员/付费资源打上 OSS 私有标记，通过 API 签名 URL 鉴权访问

// Media Uploader 组件流程

1. 用户选择文件 → 前端校验格式/大小 (< 10MB, jpg/png/webp)

2. fetch POST /api/media/upload-token → 获取 OSS STS 临时凭证

3. 前端用 STS 直传 OSS → 进度条

4. 上传完成 → fetch POST /api/media/confirm { key, contentItemId, type }

5. 服务端写入 Media 表 → 返回 mediaId + CDN URL

九、SEO 策略

- 所有资源页面 SSG: generateStaticParams 预渲染全部内容
- 动态 sitemap.xml: /sitemap.xml 路由自动生成
- 结构化数据: 资源详情页注入 JSON-LD (Article / SoftwareApplication)
- Meta 管理: 每篇 ContentItem 用 seoTitle + seoDescription + coverImage 生成 og:image
- 分类页 canonical: 防止重复内容被搜索引擎惩罚

十、实施路线图

阶段	目标	关键交付	预估工时
P1	基础搭建 首页 + 列表页 + 详情页可运行	- Prisma 完整 Schema + 种子数据 10 条 - 首页从数据库渲染 (替代 site-data.ts) - 分类列表页 SSG - 资源详情页 /resource/[slug] - 搜索接口 /api/search	2 周

P2	认证 + 会员系统	• next-auth 集成 (邮箱密码 + OTP) • middleware 鉴权路由守卫 • 会员中心: 收藏/下载记录/历史 • 下载鉴权接口 • 积分 + 签到	2 周
P3	后台管理系统	• Admin 布局 + 侧栏导航 • 资源 CRUD (单条 + 批量) • 图片上传 (OSS 直传) • 分类树管理 • 用户/会员管理	3 周
P4	迁移 + 优化	• 旧站数据迁移脚本 • 批量导入 TXT 解析工具 • 性能优化 (ISR 缓存) • 安全审计 + 渗透测试 • 旧站域名切换	2 周

十一、技术决策记录

决策	选择	理由
富文本编辑器	TipTap (不依赖内容的场景用 Markdown)	用户实际并不需要复杂富文本——标题+图片+下载链接即可。简单场景用 textarea + Markdown; 如需富文本, TipTap 比 Quill/Plate 更可控。
图片存储	阿里云 OSS	国内访问速度快, STS 直传免服务器中转, 成本低
文件上传组件	自研 MediaUploader	需求简单 (拖拽+进度条+缩略图), 不引入额外依赖
认证库	next-auth (Auth.js)	原生支持 Next.js, 内置 CSRF + Session 安全, 可接微信/QQ 登录
速率限制	Upstash Redis Ratelimit	Serverless 友好, 无需运维 Redis
部署	Vercel / 阿里云 ECS + Docker	小流量 Vercel 免费够用; 大流量切 ECS
是否需要 CMS	自研后台优先, 后续可选 Payload	现有需求用自研表单即可覆盖; Payload 适合扩展到更复杂的 CMS 场景时引入

十二、前端数据流：何时用 Server Component vs Client Component

场景	策略	原因
首页资源列表	Server Component + 直接 Prisma 查询	无需交互，服务端直接查库渲染
搜索框输入	Client Component + debounce + fetch	需要实时响应输入
收藏按钮	Client Component + Server Action	需要乐观更新 UI
后台表单提交	Client Component + fetch POST	需要表单状态管理
下载链接获取	Server Action	网盘密码不应暴露在前端 JS bundle

十三、项目启动 CMD

```
# 1. 数据库初始化和种子数据
npx prisma db push
npx prisma db seed

# 2. 启动开发服务器
npm run dev

# 3. 首次需要生成 Prisma Client
npx prisma generate

# 4. 如果要生成 slug（旧站迁移后）
npm run migrate:slugs

# 5. 构建 SSG 页面
npm run build
```